리눅스 파일 시스템 정리

첨부 필기 기반 · inode / 메타데이터 / 하드 링크 / 심볼릭 링크

01 파일은 어떻게 저장될까?

파일을 이해할 때는 이름, 관리 정보, 실제 내용을 나눠서 생각하면 쉽다. 다음 구조는 일반적인 inode 기반 리눅스 파일 시스템을 단순화한 것이다.

> 디렉터리의 파일 이름 → inode 번호 → inode → 실제 데이터 블록

구성 요소	역할	예시
디렉터리 엔트리	파일 이름과 inode 번호를 연결	a.txt → inode 20
inode	파일의 메타데이터와 데이터 위치 정보를 관리	권한, 소유자, 크기, 링크 수 등
데이터 블록	파일의 실제 내용을 저장	hello world

02 메타데이터와 inode

메타데이터는 “데이터를 설명하는 정보”다. 파일 내용이 hello world라면, 파일 크기·권한·소유자·시간 정보 등은 그 파일을 관리하는 메타데이터에 해당한다.

inode에는 파일 종류, 권한, 소유자와 그룹, 크기, 시간 정보, 하드 링크 수, 데이터 위치 정보 등이 들어간다. 파일 이름은 inode 자체가 아니라 디렉터리 엔트리에 저장된다.

inode 번호는 같은 파일 시스템 안에서 inode를 식별하는 번호다. 서로 다른 파일 시스템에서는 같은 번호가 나올 수 있다.

03 디렉터리도 파일이다

디렉터리도 자신의 inode를 갖는다. 디렉터리의 데이터에는 내부 항목의 이름과 inode 번호를 연결하는 정보가 들어간다. 필기의 “Directory table”은 이 연결 관계로 이해하면 된다.

기억할 문장: 파일 이름은 inode를 찾기 위한 이름이고, 실제 내용은 데이터 블록에 있다.

하드 링크와 심볼릭 링크

같은 파일을 가리키는 방식이 서로 다르다.

04 하드 링크: 같은 inode를 공유하는 이름

하드 링크는 기존 파일과 같은 inode를 가리키는 디렉터리 엔트리를 추가한다. 파일 내용을 복사하지 않으며, 두 이름 사이에 특별한 “원본과 복사본”의 구분도 없다.

```
a.txt → inode 20 → 실제 데이터
hard-link.txt → inode 20 → 같은 실제 데이터
```

어느 이름으로 내용을 수정해도 같은 파일의 내용이 바뀐다. a.txt라는 이름을 삭제해도 hard-link.txt가 남아 있으면 파일에 계속 접근할 수 있다.

05 심볼릭 링크: 대상 경로를 저장하는 파일

심볼릭 링크(소프트 링크)는 자신의 inode를 가진 별도 파일이다. 여기에 대상의 경로 문자열을 저장하고, 접근할 때 그 경로를 따라간다. 대상 inode 번호를 직접 저장하는 방식이 아니다.

```
soft-link.txt → 자신의 inode 30 → 경로 "a.txt"
경로 a.txt를 해석 → inode 20 → 실제 데이터
```

a.txt가 사라지면 링크 자체는 남아 있어도 대상에 접근할 수 없다. 이를 끊어진 링크 또는 dangling link라고 한다. 짧은 경로는 별도 데이터 블록 없이 inode 안에 저장될 수도 있다.

비교	하드 링크	심볼릭 링크
생성	ln a.txt hard-link.txt	ln -s a.txt soft-link.txt
inode	대상과 동일	대상과 별도
저장하는 관계	같은 inode를 가리킴	대상 경로를 저장
대상 이름 삭제	다른 하드 링크로 접근 가능	그 경로가 없어지면 끊어짐
다른 파일 시스템	일반적으로 불가	가능
디렉터리 연결	일반적으로 불가	가능

심볼릭 링크를 만들어도 대상 파일의 하드 링크 수는 증가하지 않는다.

명령어로 확인하기

새 실습 폴더에서 순서대로 실행해 보기

06 생성과 inode 확인

```
mkdir link-practice
cd link-practice
printf 'hello world\n' > a.txt
ln a.txt hard-link.txt
ln -s a.txt soft-link.txt
ls -li
```

ls -li의 맨 앞 숫자는 inode 번호다. a.txt와 hard-link.txt는 같고 soft-link.txt는 다르다. 권한 표시 뒤의 숫자는 하드 링크 수다.

```
20 -rw-r--r-- 2 user group ... a.txt
20 -rw-r--r-- 2 user group ... hard-link.txt
30 lrwxrwxrwx 1 user group ... soft-link.txt -> a.txt
```

위 출력의 번호·권한·사용자명은 설명을 위한 예시다. 실제 값은 환경에 따라 달라진다.

07 이름을 삭제하면?

```
rm a.txt
cat hard-link.txt
cat soft-link.txt
ls -l soft-link.txt
readlink soft-link.txt
```

1. cat hard-link.txt: hello world가 출력된다.
2. cat soft-link.txt: 대상 경로가 없어 오류가 발생한다.
3. ls -l: soft-link.txt → a.txt라는 링크는 여전히 보인다.
4. readlink: 링크에 저장된 a.txt라는 경로가 출력된다.

실제 파일 공간은 하드 링크 수가 0이고, 파일을 열어 둔 프로세스 등의 참조도 모두 없어져야 회수된다. 단순히 최초 이름 하나를 지웠다고 바로 데이터가 없어지는 것은 아니다.

08 시험 전에 꼭 기억할 점

- 하드 링크 = 같은 inode를 공유하는 여러 이름.
- 심볼릭 링크 = 대상 경로를 저장하는 별도 파일.
- 링크가 보인다고 대상 파일까지 존재하는 것은 아니다.
- 상대 경로 심볼릭 링크는 링크가 있는 디렉터리를 기준으로 해석한다.
- ln은 영문 소문자 l과 n이다. 대문자 I로 시작하지 않는다.